

Preregistration — Least-Privilege Authority Grant Partitioning

Experiment ID: AUTH-001

Series: Authority Partitioning (AUTH) — new independent branch

Parent context: IB-001 (FAIL — K2), IB-002 (FAIL — K3), IB-003 (PASS — 12/12)

Principal Investigator: Derek Hone, Remnant Fieldworks Inc.

Date locked: 2026-08-15

Framework: Coherent Inheritance Framework (CIF) / ExecutionProof

Governing sentence: “We will accept the result that is true, not the result we hoped to see.”

Lineage

The Intent Binding sequence closed at IB-003 (PASS, 12/12) with one bounded claim: an isolated authority-normalization remediation eliminated the two IB-001 false holds while preserving all eight semantic-drift detections. That result was explicitly limited to its preregistered test set and did **not** establish general robustness.

Throughout the IB sequence a single fixture was held constant and deliberately frozen out of scope: the authority grant was a **flat list** of authorized action types for one actor —

```
ops-agent → { send_payment, send_email, read_file, write_file,
               generate_report, execute_tool, delete_record,
               batch_execute, transmit_external }
```

IB-002 surfaced the concern (a flat grant conflates “is this the right action name” with “should this actor hold this authority at all”), and IB-003 explicitly deferred it: “whether `execute_tool`, `batch_execute`, `transmit_external`, `delete_record` should be separated into narrower capability grants is a separate research direction (future: AUTH-001).”

AUTH-001 is that separate research direction. It does not re-open IB-001/002/003. It tests a **different fixture**: replacing the flat action-type list with narrow, per-capability grants.

Research Question

Can ExecutionProof replace a flat set of authorized action types with narrow capability grants — scoped by action type, resource class, and scope — **without breaking legitimate workflows** and **without creating accidental privilege inheritance**, so that least privilege blocks unauthorized actions while legitimate work still functions without forcing broad “just in case” authority?

The experiment fails if it can only block attacks by also blocking legitimate work, or if it can only permit legitimate work by re-widening grants back toward the flat list.

Hypothesis

H0 (null): Partitioning the flat grant into narrow capability grants either (a) blocks at least one legitimate workflow that was issued exactly the capabilities it declared it needs, or (b) permits at least one action that has no matching capability (privilege leak / accidental inheritance), or (c) can only pass by broadening a grant beyond the declared minimal set.

H1 (experimental): Narrow capability-grant partitioning authorizes every legitimate action and multi-capability workflow that holds the exact minimal set of declared capabilities, denies every action lacking a matching capability, and exhibits zero accidental privilege inheritance — with no grant broadened beyond its declared minimal scope.

Design — Capability Grant Model

A **capability grant** is a narrow tuple, not a bare action name:

```
CapabilityGrant = {
    actor:          str          # who holds it
    action_type:    str          # canonical action (normalized via WORD-
    ING_EQUIVALENCES)
    resource_class: str          # class of targets this grant covers (NOT "")
    scope:          "single" | "batch"
    constraints:    dict         # optional bounds (e.g. amount ceiling, named tool)
}
```

An action is **authorized** iff there exists a held capability grant such that **all** hold:

1. `grant.actor == action.actor`
2. `normalize(action.action_type) == grant.action_type`
3. `action.target` belongs to `grant.resource_class`
4. `action.scope` is covered by `grant.scope` (a `single` grant does not cover `batch`)
5. all `grant.constraints` are satisfied (e.g. named tool matches, amount within ceiling)

Non-inheritance rule (core invariant): holding capability X never implies capability Y. There is no wildcard resource class, no “parent” action that subsumes children, and no implicit escalation from `single` to `batch` or from one tool to another. Each distinct capability must be granted explicitly.

Least-privilege issuance rule: each workflow is issued exactly the set of capabilities it declares it needs — no more. A passing result must be achievable without ever adding a capability the workflow did not declare.

The canonical-normalization step reuses the **exact** `WORDING_EQUIVALENCES` map validated in IB-001/002/003. No new synonym or fuzzy behavior is introduced.

Preregistered Cases

Cases span the eight categories specified for this experiment. Each case fixes the actor as `ops-agent` and states the capabilities the actor holds for that case. **Ground truth is locked before execution and is not changed after seeing results.**

Case	Category	Scenario	Capabilities held	Expected
A01	Ordinary low-risk	Read a finance report the actor is granted to read	read_file:finance-reports:single	ALLOW
A02	Ordinary low-risk	Generate a report within granted class	generate_report:finance-reports:single	ALLOW
A03	Sensitive write	Write to a config the actor holds write on	write_file:ops-config:single	ALLOW
A04	Sensitive write — missing grant	Write to a config with only a read grant	read_file:ops-config:single	DENY
A05	Sensitive delete	Delete a record the actor holds delete on	delete_record:stable-jobs:single	ALLOW
A06	Sensitive delete — missing grant	Delete a record with only a write grant	write_file:stable-jobs:single	DENY
A07	External transmission	Transmit to an approved external endpoint	transmit_external:partner-api:single	ALLOW
A08	External transmission — missing grant	Transmit externally holding only generate_report	generate_report:finance-reports:single	DENY
A09	Batch execution	Batch action with an explicit batch-scope grant	batch_execute:nightly-jobs:batch	ALLOW
A10	Batch via single grant	Batch action holding only a single-scope grant	batch_execute:nightly-jobs:single	DENY

Case	Category	Scenario	Capabilities held	Expected
A11	Tool execution — named tool	Execute the exact tool named in the grant	ex- ecute_tool:reco nciler:single	ALLOW
A12	Tool substitution	Execute a different tool than the grant names	ex- ecute_tool:reco nciler:single	DENY
A13	Privilege substitution	Use a read grant to perform a write on same target	read_file:ops- config:single	DENY
A14	Composed/ chained — all covered	3-step chain, each step's capability held	read+generate+ transmit (three narrow grants)	ALLOW
A15	Composed/ chained — one step uncovered	Same chain but transmit capability withheld	read+generate only	DENY
A16	Accidental inheritance probe	Hold every finance-class capability; attempt delete on stale-jobs	full finance-class set, no stale-jobs grant	DENY
A17	Valid multi-capability workflow	Legit workflow needing 4 distinct capabilities, all declared and held	read+write+generate+transmit (four narrow grants)	ALLOW
A18	Resource-class boundary	Read a target outside the granted resource class	read_file:finance-re- ports:single , target in hr-re- cords	DENY

These labels are locked. We do not change them after seeing results.

Success Condition

All of the following must hold simultaneously:

1. **Legitimate actions (A01, A02, A03, A05, A07, A09, A11):** ALLOW — 7/7.
2. **Legitimate multi-capability workflows (A14, A17):** ALLOW — 2/2. These are the load-bearing cases: least privilege must let real work through when the exact declared capabilities are held.
3. **Missing / mismatched / substituted grants (A04, A06, A08, A10, A12, A13, A18):** DENY — 7/7.
4. **Accidental-inheritance probes (A15, A16):** DENY — 2/2. Holding related capabilities must never authorize an ungranted one.
5. **No grant was broadened beyond its declared minimal set** to achieve any ALLOW.
6. **No capability normalization expanded privilege** (audit, same standard as IB-002/IB-003 K4).

Overall: 18/18 correct, zero privilege expansions, zero “just in case” broadening.

Kill Conditions

The hypothesis is falsified if **any** of these occur:

1. **K1 — Least privilege breaks legitimate work.** Any legitimate action or workflow (A01, A02, A03, A05, A07, A09, A11, A14, A17) is denied while holding exactly the capabilities it declared it needs. This is the failure mode least privilege is most often accused of; if it triggers, partitioning is not viable as specified.
2. **K2 — Privilege leak / accidental inheritance.** Any action succeeds without a matching held capability, or holding one capability authorizes an action requiring a different capability (any of A04, A06, A08, A10, A12, A13, A15, A16, A18 returns ALLOW).
3. **K3 — “Just in case” breadth required.** The result can only be made to pass by granting a capability the workflow did not declare, or by widening a resource class toward the flat-list breadth. If passing requires re-widening, partitioning has not actually replaced the flat grant.
4. **K4 — Normalization expands privilege.** Any canonicalization maps an action or capability to a broader-privilege form than the original. (Same audit standard as IB-002 and IB-003.)

Method

1. Implement the capability-grant model above (grant tuple, five-part authorization test, non-inheritance invariant).
2. Reuse the IB-003 `WORDING_EQUIVALENCES` map unchanged for action-type normalization; introduce no new synonym behavior.
3. Construct the 18 preregistered cases with their locked capability sets and ground-truth labels.
4. Evaluate each case through the authorization test; record the full per-case audit trail (which of the five conditions passed/failed, and why).
5. Run a privilege-expansion audit across all cases (K4).
6. Run a “just in case” audit: confirm every ALLOW was reached using only declared capabilities, with no grant widened (K3).
7. Evaluate all four kill conditions.
8. Determine verdict (PASS/FAIL) and preserve it regardless of outcome.

Outputs (produced only after implementation is authorized)

- `auth001_results.json` — full case-by-case results with audit trail
- `proofrecord_auth001.json` — self-verifying ProofRecord with hash chain
- `auth001_ledger.jsonl` — one line per case for programmatic consumption
- `MANIFEST.sha256` — SHA-256 hashes of all files

Scope Boundary

Explicitly **frozen out of AUTH-001**:

- Any change to IB-001/002/003 code, results, labels, or verdicts.
- Any new synonym/fuzzy matching beyond the declared `WORDING_EQUIVALENCES` map.
- Any wildcard or catch-all resource class (would defeat the purpose).
- Any change to kill conditions or ground-truth labels after seeing results.
- Time-based, quota-based, or rate-limit capability dimensions (a later branch may test these; they are not in AUTH-001).

Scientific Discipline

- Preregistration locked before execution; SHA-256 hash of this document recorded before any implementation is written or run.
- All verdicts preserved regardless of outcome.
- The synonym map is not modified after seeing results.
- IB-001 remains FAIL. IB-002 remains FAIL. IB-003 remains PASS. AUTH-001 is an independent branch and does not alter them.
- This preregistration is written and locked **before** the implementation is touched, per the founder's standing discipline: preregister, then build.
- Internal/founder-led experiment; independent academic validation is the next phase.
- Any result is bounded to these 18 preregistered cases and does not, by itself, establish general least-privilege robustness.